

EchoConnector - Team Work Breakdown (5 Members)

Team Structure

Total Build Time: 4 days (hackathon)

Team Size: 5 members

Key Principle: Member 5 (Presentation Lead) has 50% lighter workload



Member 1: Production Watchman Lead (Backend - Heavy)

Role: Build the core incident detection loop

Estimated Time: 16-18 hours over 4 days

Day 1 (4 hours)

- [] Set up AWS account and CDK project
- [] Create EventBridge rule (60-second schedule)
- [] Create basic Lambda skeleton that logs "Hello from Watchman"
- [] Test EventBridge → Lambda trigger
- [] Set up CloudWatch Logs for debugging

Deliverable: Lambda executing every 60 seconds

Day 2 (5 hours)

- [] Implement CloudWatch alarms polling in Lambda
- [] Add logic to detect ALARM state
- [] Parse alarm metadata (service, resource, metric)
- [] Create test alarm for RDS high CPU
- [] Test alarm detection flow

Deliverable: Lambda detects when alarms fire

Day 3 (4 hours)

- [] Integrate with Member 2's Bedrock function
- [] Pass alarm data + runbook context to Bedrock
- [] Parse Bedrock response (root cause + fixes)
- [] Handle errors and retries

- ☐ Add logging for debugging

Deliverable: End-to-end flow from alarm detection to AI analysis

Day 4 (3 hours)

- ☐ Integration testing with all components
- ☐ Fix bugs discovered during testing
- ☐ Add error handling edge cases
- ☐ Performance optimization
- ☐ Final testing with Member 5's demo script

Deliverable: Production-ready Watchman loop

Handoff Points

- **To Member 2:** Alarm data format, when to call Bedrock
- **To Member 3:** Incident data format for DynamoDB
- **To Member 4:** When to trigger Slack notification
- **To Member 5:** How to trigger test alarms for demo



Member 2: Bedrock Integration Lead (AI/ML - Heavy)

Role: Build AI analysis engine with Bedrock

Estimated Time: 16-18 hours over 4 days

Day 1 (4 hours)

- ☐ Research Bedrock API documentation
- ☐ Create IAM role with Bedrock permissions
- ☐ Write basic Lambda function to call Bedrock
- ☐ Test simple prompt: "Explain what CloudWatch is"
- ☐ Verify Bedrock responses are working

Deliverable: Working Bedrock API integration

Day 2 (5 hours)

- ☐ Design system prompt for incident analysis
- ☐ Create prompt template with placeholders
- ☐ Implement structured output parsing (JSON)
- ☐ Test with sample alarm data
- ☐ Refine prompts based on output quality

Deliverable: Bedrock returns structured incident analysis

Day 3 (5 hours)

- [] Integrate S3 runbook reading (from Member 3)
- [] Add runbook context to Bedrock prompts
- [] Implement guardrails (if time permits)
- [] Test with different alarm types (RDS, Lambda, ECS)
- [] Handle edge cases (timeout, invalid JSON)

Deliverable: Context-aware AI analysis with runbooks

Day 4 (3 hours)

- [] Fine-tune prompts for better suggestions
- [] Add confidence scores to suggestions
- [] Test with all demo scenarios
- [] Optimize token usage (cost reduction)
- [] Document prompt engineering decisions

Deliverable: Production-quality AI analysis

Key Functions to Build

```
def analyze_with_bedrock(alarm_data, runbook_context):
```

```
    """
```

```
    Input: Alarm details + runbook text
```

```
    Output: {
```

```
        "root_cause": "...",
```

```
        "impact": "...",
```

```
        "suggested_fixes": [
```

```
            {
```

```
                "title": "...",
```

```
                "steps": [...],
```

```
                "confidence": "high",
```

```
                "risk": "low"
```

```
            }
```

```
        ]
```

```
    }
```

```
    """
```

```
    pass
```

Handoff Points

- **From Member 1:** Alarm data format
 - **From Member 3:** Runbook text format
 - **To Member 1:** Analysis response format
 - **To Member 5:** Sample prompts for presentation
-

Member 3: Data & Storage Lead (Backend - Heavy)

Role: Build S3 runbooks system + DynamoDB storage

Estimated Time: 14-16 hours over 4 days

Day 1 (3 hours)

- ☐ Create S3 bucket: `echo-runbooks`
- ☐ Design runbook directory structure
- ☐ Write 3 basic runbooks (RDS, Lambda, ECS)
- ☐ Upload runbooks to S3
- ☐ Test S3 read access from Lambda

Deliverable: S3 bucket with runbooks

Sample Runbooks to Create:

1. `aws/rds/high-cpu.md` - RDS CPU troubleshooting
2. `aws/lambda/high-errors.md` - Lambda error handling
3. `aws/ecs/task-failures.md` - ECS task issues

Day 2 (4 hours)

- ☐ Create DynamoDB table: `echo-incidents`
- ☐ Design schema (incidentId, timestamp, severity, etc.)
- ☐ Write Lambda helper functions for DynamoDB
- ☐ Test write operations
- ☐ Add GSI for querying by status/severity

DynamoDB Schema:

```
{
  "incidentId": "INC-2026-03-11-001",
  "timestamp": "2026-03-11T14:22:00Z",
  "severity": "CRITICAL",
  "service": "RDS",
  "resource": "orders-db-prod",
  "alarmName": "rds-high-cpu",
  "aiAnalysis": {...},
  "status": "new",
  "createdAt": 1234567890
}
```

Day 3 (4 hours)

- ☐ Implement runbook retrieval logic

- [] Add keyword matching for relevant runbooks
- [] Create function to save incidents to DynamoDB
- [] Add error handling (S3 access denied, etc.)
- [] Test with Member 2's Bedrock integration

Key Functions:

```
def get_relevant_runbook(alarm_data):
    """Returns runbook text based on alarm type"""
    pass
```

```
def save_incident(incident_id, alarm_data, analysis):
    """Saves incident to DynamoDB"""
    pass
```

```
def get_incidents(status=None, severity=None):
    """Query incidents for reporting"""
    pass
```

Day 4 (3 hours)

- [] Write 2 more detailed runbooks based on testing
- [] Add incident querying capabilities
- [] Create helper script to bulk upload runbooks
- [] Test data persistence end-to-end
- [] Document runbook format for team

Deliverable: Complete data layer

Handoff Points

- **To Member 2:** Runbook text format, retrieval function
- **To Member 1:** DynamoDB save function
- **To Member 5:** Sample runbooks for demo, query functions

Member 4: Slack/Teams Integration Lead (Full-Stack - Medium)

Role: Build notification system + interactive messages

Estimated Time: 14-16 hours over 4 days

Day 1 (3 hours)

- [] Create Slack workspace (or use existing)

- [] Create #incidents channel
- [] Set up Incoming Webhook in Slack
- [] Store webhook URL in AWS Secrets Manager
- [] Test sending "Hello World" to Slack

Deliverable: Working Slack webhook

Day 2 (5 hours)

- [] Design rich Slack message layout (blocks)
- [] Implement message formatting function
- [] Add severity emoji mapping
- [] Create message templates for each severity
- [] Test different message variations

Message Components:

- Header with severity
- Incident metadata (ID, time, service)
- AI analysis section
- Suggested fixes (2-3 fixes)
- Action buttons (View Console, Mark Resolved)

Day 3 (4 hours)

- [] Integrate with Member 1's Lambda
- [] Fetch webhook URL from Secrets Manager
- [] Send notifications when incidents detected
- [] Handle Slack API errors (rate limits, etc.)
- [] Add retry logic for failed notifications

Key Function:

```
def send_slack_notification(incident_id, alarm_data, analysis):
    """
    Sends rich formatted message to Slack
    """
    webhook_url = get_secret('echo/slack-webhook')
    message = format_slack_message(incident_id, alarm_data, analysis)
    response = requests.post(webhook_url, json=message)
    return response.status_code == 200
```

Day 4 (3 hours)

- [] Polish message formatting
- [] Add timestamps and timezone handling
- [] Test with all demo scenarios
- [] Add support for Teams webhook (if time permits)

- [] Document webhook setup process

Deliverable: Beautiful Slack notifications

Handoff Points

- **From Member 1:** When to send notifications
 - **From Member 2:** Analysis data to display
 - **To Member 5:** Screenshots of Slack messages for slides
-

Member 5: Presentation & Environment Lead (DevOps - Light)

Role: Demo preparation, CDK deployment, presentation

Estimated Time: 8-10 hours over 4 days (50% lighter!)

Day 1 (2 hours)

- [] Set up AWS account (free tier)
- [] Configure AWS CLI and credentials
- [] Install CDK toolkit: `npm install -g aws-cdk`
- [] Bootstrap CDK: `cdk bootstrap`
- [] Create GitHub repo for team

Deliverable: Development environment ready

Day 2 (2 hours)

- [] Create CDK stack skeleton
- [] Add IAM roles and permissions
- [] Deploy first version with placeholder Lambda
- [] Test deployment process
- [] Document deployment steps

CDK Stack:

// Minimal stack with all resources

- EventBridge rule
- Lambda function (Watchman)
- DynamoDB table
- S3 bucket
- IAM roles
- Secrets Manager secret

Day 3 (2 hours)

- ☐ Write demo script (step-by-step)
- ☐ Create test CloudWatch alarms for demo
- ☐ Prepare demo data (fake incidents)
- ☐ Test complete demo flow
- ☐ Time the demo (should be ~5 minutes)

Demo Script:

1. Show EchoConnector architecture diagram (30s)
2. Trigger test alarm (10s)
3. Watch Lambda logs in CloudWatch (30s)
4. Show Slack notification arrive (30s)
5. Explain AI analysis (60s)
6. Show suggested fixes (60s)
7. Check DynamoDB for saved incident (30s)
8. Q&A (60s)

Day 4 (3 hours)

- ☐ Create presentation slides (10-12 slides)
- ☐ Record demo video (backup in case live demo fails)
- ☐ Practice presentation 3 times
- ☐ Prepare answers to likely questions
- ☐ Final deployment to clean AWS account

Presentation Outline:

1. **Problem** (1 slide) - AWS incidents are complex
2. **Solution** (1 slide) - EchoConnector = AI-powered observability
3. **Architecture** (2 slides) - Show application diagram
4. **Three Loops** (3 slides) - Watchman, Chat, Cost Optimizer
5. **Live Demo** (5 minutes) - Show real incident detection
6. **Impact** (1 slide) - Benefits and savings
7. **Future** (1 slide) - Roadmap
8. **Q&A** (2 minutes)

Deliverable: Polished demo + presentation

Additional Responsibilities

- ☐ Keep team on schedule (daily standup)
- ☐ Resolve AWS account issues
- ☐ Deploy final version before presentation
- ☐ Be backup for any team member who gets stuck

Handoff Points

- **From Everyone:** Code to deploy via CDK
 - **From Member 1:** Test alarm creation commands
 - **From Member 4:** Slack webhook setup instructions
 - **To Everyone:** Deployment credentials, demo timeline
-



Work Distribution Summary

Member	Role	Total Hours	Work Intensity
Member 1	Production Watchman	16-18h	Heavy
Member 2	Bedrock Integration	16-18h	Heavy
Member 3	Data & Storage	14-16h	Heavy
Member 4	Slack Integration	14-16h	Medium
Member 5	Presentation & DevOps	8-10h	Light 

Total Team Hours: 68-78 hours over 4 days

Average per person: 13-16 hours

Member 5 (Presenter): 50% less work than others



Daily Sync Schedule

Daily Standup (15 minutes, 9:00 AM)

Each member shares:

1. What I completed yesterday
2. What I'm working on today
3. Any blockers

Member 5 (Lead) tracks:

- Overall progress vs timeline
- Integration points between members
- Risk areas that need attention

Daily Integration Points

Day 1 End:

- Member 5: AWS environment ready

- Member 1: Lambda skeleton deployed
- Member 3: S3 bucket + runbooks created
- Member 4: Slack webhook working

Day 2 End:

- Member 2: Bedrock returning analysis
- Member 3: DynamoDB schema finalized
- Member 1 + 2: First integration test (alarm → Bedrock)

Day 3 End:

- All members: End-to-end test (alarm → Bedrock → DynamoDB → Slack)
- Member 5: Demo script ready

Day 4:

- Morning: Final integration testing
- Afternoon: Polish + presentation prep



Risk Mitigation

High-Risk Items

1. Bedrock API Access (Member 2)

- **Risk:** AWS account might not have Bedrock enabled
- **Mitigation:** Member 5 enables Bedrock on Day 1
- **Backup:** Use mocked responses if Bedrock unavailable

2. Integration Between Members (Everyone)

- **Risk:** Different data formats cause integration failures
- **Mitigation:** Define data contracts upfront (see below)
- **Backup:** Member 5 mediates conflicts

3. Demo Failure (Member 5)

- **Risk:** Live demo fails during presentation
- **Mitigation:** Record backup video on Day 3
- **Backup:** Show architecture + video instead of live

Data Contracts (Agreed Day 1)

Alarm Data Format:

```
{
```

```
"AlarmName": "rds-high-cpu",
"Namespace": "AWS/RDS",
"MetricName": "CPUUtilization",
"StateValue": "ALARM",
"Threshold": 80,
"Dimensions": [{"Name": "DBInstanceIdentifier", "Value": "orders-db-prod"}]
}
```

Bedrock Analysis Format:

```
{
  "root_cause": "RDS CPU at 94% due to unindexed query...",
  "impact": "Checkout degradation, 15% timeout rate",
  "confidence": "high",
  "suggested_fixes": [
    {
      "title": "Add index on orders.status",
      "steps": ["CREATE INDEX CONCURRENTLY..."],
      "risk": "low",
      "confidence": "high",
      "expected_result": "CPU drops to 30%"
    }
  ]
}
```

Incident Storage Format:

```
{
  "incidentId": "INC-2026-03-11-001",
  "timestamp": "2026-03-11T14:22:00Z",
  "severity": "CRITICAL",
  "service": "RDS",
  "resource": "orders-db-prod",
  "aiAnalysis": {...}, # Bedrock response
  "status": "new"
}
```



Communication Channels

Slack/Teams Channel: [#echo-hackathon](#)

GitHub Repo Structure:

echo-connector/

— infrastructure/	# Member 5 (CDK)
— lambda/	
— watchman/	# Member 1
— bedrock/	# Member 2
— shared/	# Member 3 (helpers)
— runbooks/	# Member 3 (S3 content)
— slack/	# Member 4 (formatting)
— presentation/	# Member 5 (slides)
— README.md	

Daily Commits: Each member commits EOD with tag `[Day1]`, `[Day2]`, etc.

Success Criteria

Minimum Viable Demo (Must Have)

-  EventBridge triggers Lambda every 60s
-  Lambda detects CloudWatch alarm
-  Bedrock analyzes incident with AI
-  Incident saved to DynamoDB
-  Slack notification sent with rich formatting
-  Live demo works end-to-end

Nice to Have (If Time Permits)

-  Multiple runbooks (>3)
-  Guardrails on Bedrock
-  Teams webhook support
-  Cost Optimizer loop (second agent)
-  Interactive Slack buttons

Judging Criteria Focus

1. **Innovation** - AI-powered incident analysis (Member 2's prompts)
 2. **AWS Integration** - Uses 8+ AWS services (Everyone)
 3. **Completeness** - Works end-to-end (Integration success)
 4. **Presentation** - Clear demo + slides (Member 5)
 5. **Real-world Impact** - Solves actual pain point (Pitch)
-



Tips for Success

For Member 1 (Watchman Lead)

- Start simple: just log alarms before adding complexity
- Use CloudWatch Logs extensively for debugging
- Test with manual alarm triggers first

For Member 2 (Bedrock Lead)

- Spend time on prompt engineering - it's 80% of quality
- Start with simple prompts, add complexity iteratively
- Keep token count low (cost optimization)

For Member 3 (Data Lead)

- Write clear, simple runbooks - don't overcomplicate
- Test S3 permissions early (common pain point)
- DynamoDB on-demand = no capacity planning needed

For Member 4 (Slack Lead)

- Use Slack Block Kit Builder for message design
- Test webhook early and often
- Beautiful messages = impressive demo

For Member 5 (Presentation Lead)

- Keep presentation simple and visual
- Practice demo 5+ times
- Have backup plan for everything



Final Checklist (Day 4 Morning)

Code:

- ☐ All functions have error handling
- ☐ Logs are clean and informative
- ☐ No hardcoded credentials
- ☐ Code is commented

Demo:

- ☐ Test alarm triggers reliably
- ☐ Slack notifications arrive in <5s
- ☐ Backup video recorded
- ☐ Demo script practiced

Presentation:

- ☐ Slides are clear and visual
- ☐ Architecture diagram is polished
- ☐ Team roles are clear
- ☐ Q&A answers prepared

Deployment:

- ☐ CDK deploys without errors
 - ☐ All AWS resources created
 - ☐ Costs are within free tier
 - ☐ Clean AWS account for demo
-

Summary

Member 1: Core incident detection (Heavy - 16-18h)

Member 2: AI/Bedrock integration (Heavy - 16-18h)

Member 3: Data & runbooks (Heavy - 14-16h)

Member 4: Slack notifications (Medium - 14-16h)

Member 5: Presentation & environment (8-10h) 

Total: 68-78 team hours over 4 days

Ready to win this hackathon! 